

Prim's Algorithm - Brief Explanation and Complexity

Objective: To find a Minimum Spanning Tree (MST) of a weighted, connected graph using Prim's algorithm.

1. Method Used in the C Code

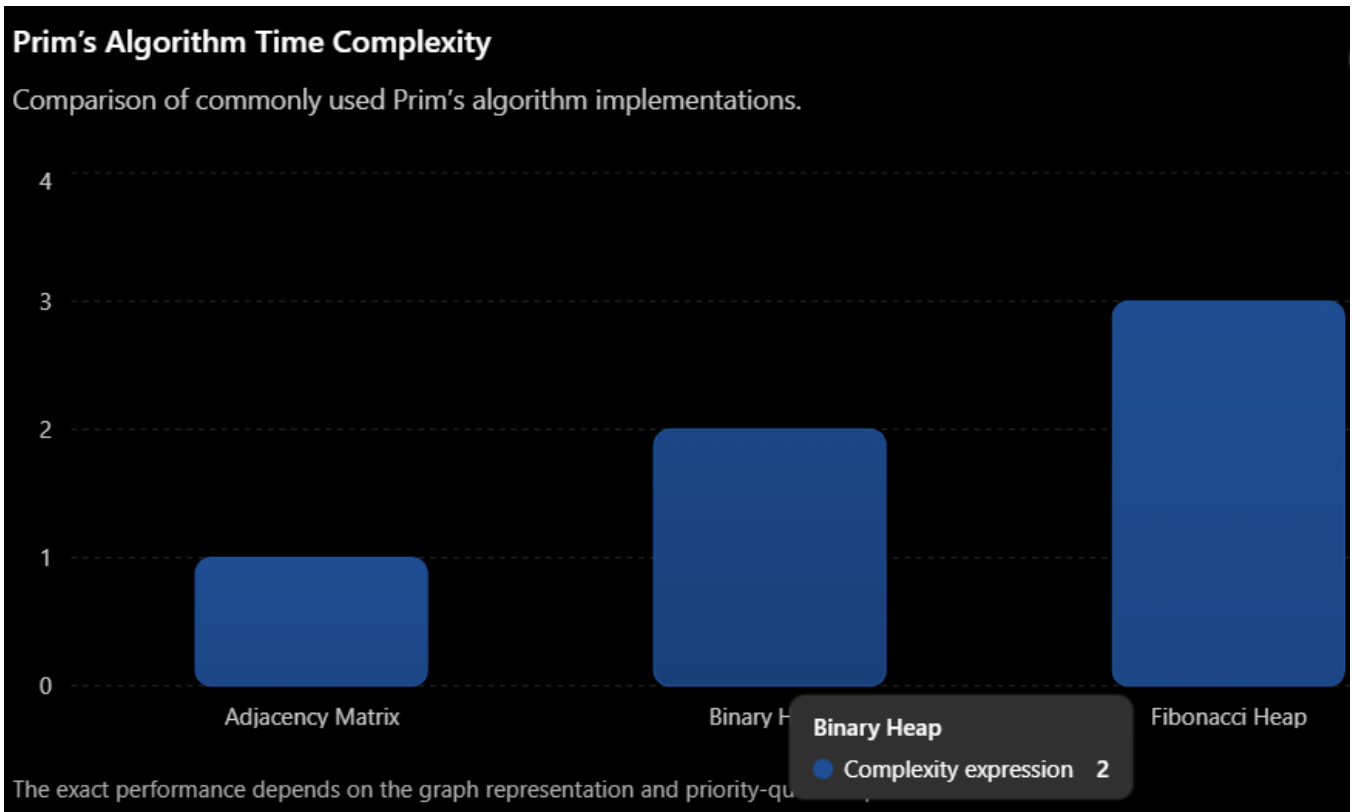
The given C program implements Prim's algorithm using an **adjacency matrix**. It starts from vertex 0 and keeps a **selected[]** array to record the vertices already included in the MST. For every step, the program checks all edges from selected vertices to unselected vertices and chooses the edge with the minimum weight. The chosen edge is printed, its cost is added to the total, and the new vertex is marked as selected. This continues until **$V - 1$** edges have been selected.

For the graph shown in the program, the MST edges obtained are 0-1 (2), 1-2 (3), 1-4 (5), and 0-3 (6), giving a **total MST cost of 16**. With an adjacency matrix and repeated scanning of vertices, this implementation has a time complexity of approximately **$O(V^2)$** .

2. Alternative Method - Priority Queue / Heap

Prim's algorithm can also be implemented using an **adjacency list and a priority queue (heap)**. Instead of checking every possible vertex pair repeatedly, the priority queue helps select the next minimum-weight edge efficiently. Using a binary heap, the commonly stated complexity is **$O(E \log V)$** . A Fibonacci heap can improve the theoretical bound to **$O(E + V \log V)$** , although it is more complex to implement and is less common in basic programs.

3. Time Complexity Chart



The chart compares three common ways of implementing Prim's algorithm: **Adjacency Matrix**, **Binary Heap**, and **Fibonacci Heap**. The bars represent different complexity expressions rather than actual measured running time. The adjacency-matrix approach is simple and suitable for small or dense graphs. A binary heap is generally more efficient

for sparse graphs, while a Fibonacci heap provides a better theoretical complexity but requires a more complicated data structure.

Implementation	Typical Time Complexity	Brief Note
Adjacency Matrix	$O(V^2)$	Simple implementation; scans vertices repeatedly.
Binary Heap + Adj. List	$O(E \log V)$	Efficient and commonly used for sparse graphs.
Fibonacci Heap	$O(E + V \log V)$	Better theoretical bound; more complex implementation.